# Restaurant Management System

# akla

## 1. Problem Description

Restaurant operations involve multiple daily tasks such as managing menus, receiving customer orders, calculating bills, applying discounts, tracking sales, and organizing table reservations. Performing these tasks manually may lead to errors, wasted time, and difficulty in maintaining accurate records.

This project aims to develop a Restaurant Management System using Python to automate these processes. The system provides an organized and efficient way to manage customer orders, store sales data, generate daily reports, and assist both customers and restaurant staff through a console-based application.

## 2. Distribution of Work Among Team Members

The project was developed collaboratively, with responsibilities divided among the team members as follows:

**Hamza mohamed:**

(load_menu)-Implemented menu loading functionality.

(save_order)-Developed order saving and receipt generation.

(calculate_total)-Implemented total bill calculation.

(apply_discount)-Designed discount application logic.

(add_menu_item)-Added functionality for inserting new items into the menu.

**Gassar khaled:**

(remove_menu_item)-Implemented menu item removal functionality.

(search_menu)-Developed menu search feature.

(get_best_selling_item)-Implemented best-selling item analysis.

(validate_order)-Implemented order validation logic.

(generate_daily_report)-Designed daily sales report generation.

**Ahmed basit:**

(get_available_tables)-Implemented table availability management.

(estimate_preparation_time)-Developed order preparation time estimation.

(register_customer)-Implemented customer registration functionality.

(notify_low_stock)-Designed low stock notification system.

(export_daily_sales_to_csv)-Implemented daily sales export to CSV files.

## 3. System Design and Solution Approach

The system is designed using a modular programming approach, where each task is handled by a dedicated function.

The menu is loaded from text files and stored using dictionaries for efficient access. Customer orders are collected interactively and saved to files to ensure data persistence.

The system calculates order totals dynamically based on menu prices and quantities, then applies discounts when applicable. Daily sales data is analyzed to identify best-selling items and generate revenue reports.

Administrative functionalities allow agents to manage menu items, monitor stock levels, and export sales data in CSV format for further analysis.

This design ensures maintainability, scalability, and clarity in the overall system structure.

## 4. Program Usage (User Manual)

Launch the program.

Select either Agent Mode or Customer Mode.

In Customer Mode, users can:

View and search the menu.

Place and validate orders.

Calculate total bills and apply discounts.

Check available tables.

In Agent Mode, staff can:

Register customers.

Manage menu items.

Estimate order preparation time.

Monitor low stock items.

Export daily sales data to CSV files.

Generate daily sales reports.

The program stores all orders and sales data automatically for future reference.